



Vehicle Weight System

Group L26

Team



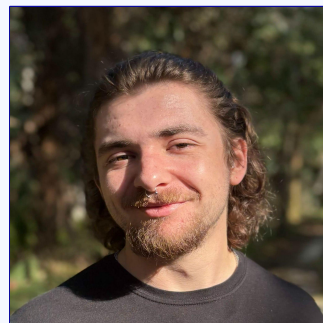
Chinmayi Achanta
PM/ML – LPR



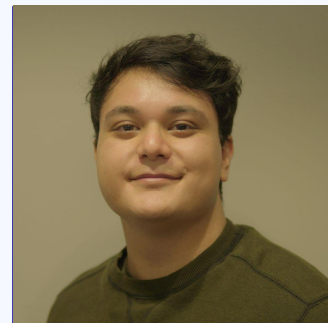
Noah Albert
Hardware/Sensor



Joseph Benno
ML – Cargo

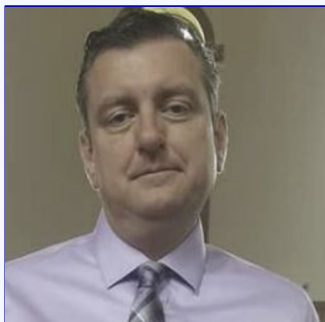


Dawson Holmes
Backend/DB

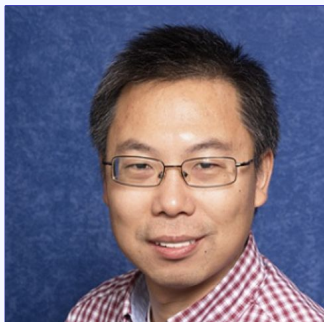


Anant Kumar
Hardware/Backend

Sponsors



Scott Piersall



Dr. Liqiang Wang

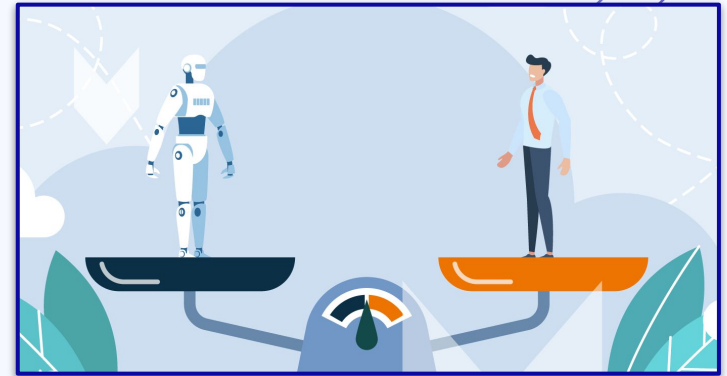


Dr. Zihang Zou



Background & Industry Context

- Landfills and material facilities process hundreds of trucks daily
- Accurate weight tracking is essential for billing and compliance
- Vehicle identification and load classification are part of the intake workflow
- Operations often run continuously, including late-night shifts
- Facilities are seeking scalable automation solutions

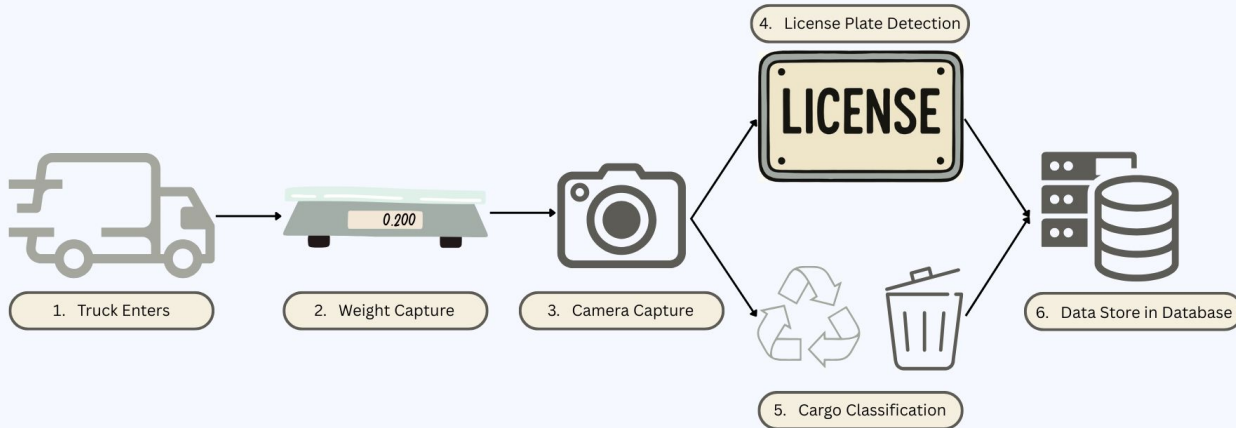


Problem Statement

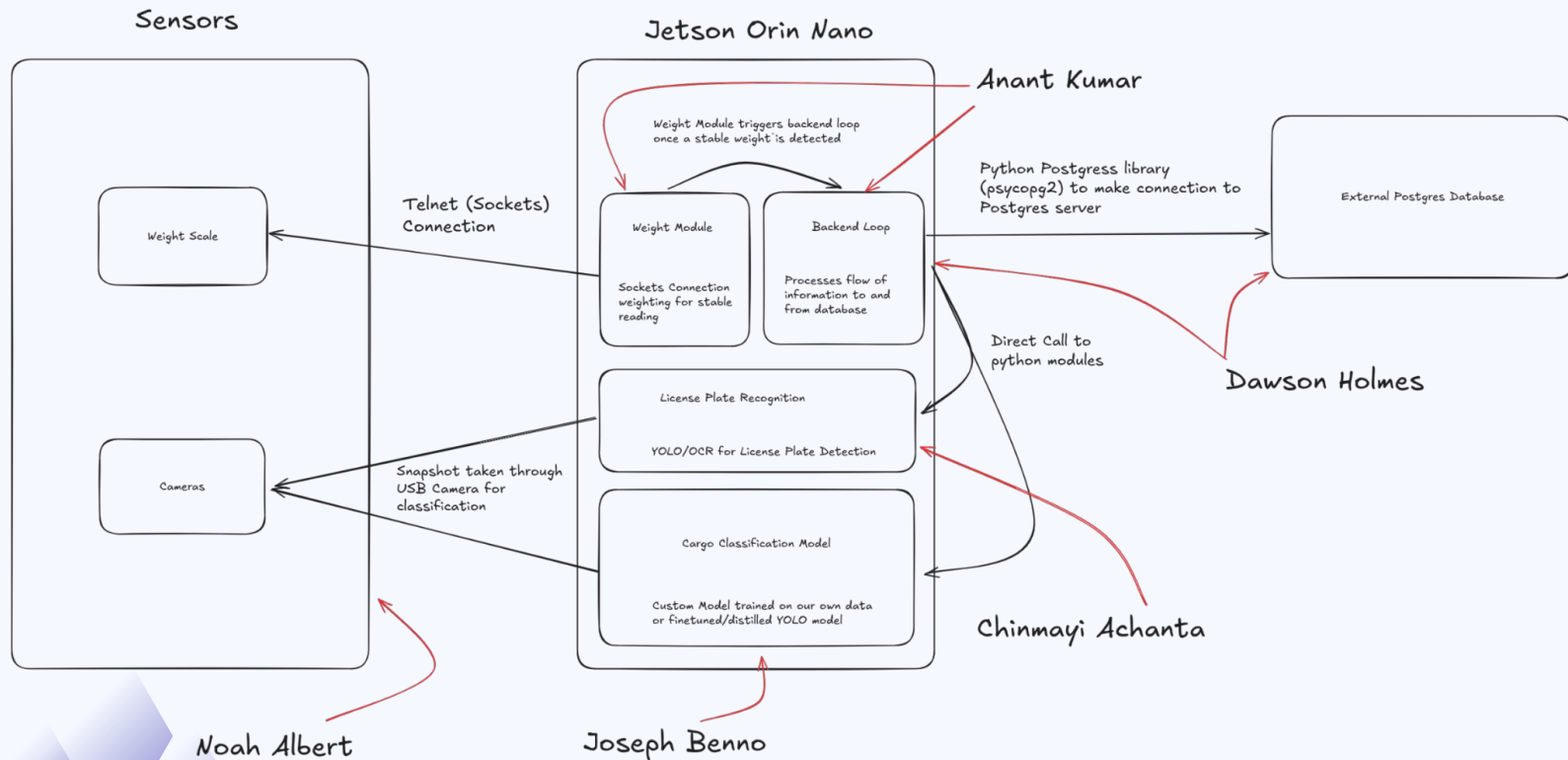
- Vehicle identification and cargo classification rely on **human verification**
- Manual steps create **bottlenecks and limit throughput**
- Data entry errors impact **billing accuracy and compliance**
- Operators work in **hazardous conditions** (weather, late-night shifts)
- Systems lack **end-to-end integration**

Proposed Solution & System Overview

- Fully automated weigh-in/weigh-out workflow
- Truck enters and stable weight is detected
- Camera captures vehicle image at weighbridge
- System identifies license plate and classifies cargo
- All data is stored automatically in a database



System Block Diagram



Technology Stack

Project Management



Machine Learning



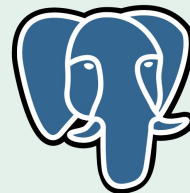
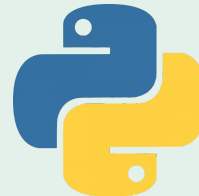
Hardware & Sensors



Backend & Database



Flask

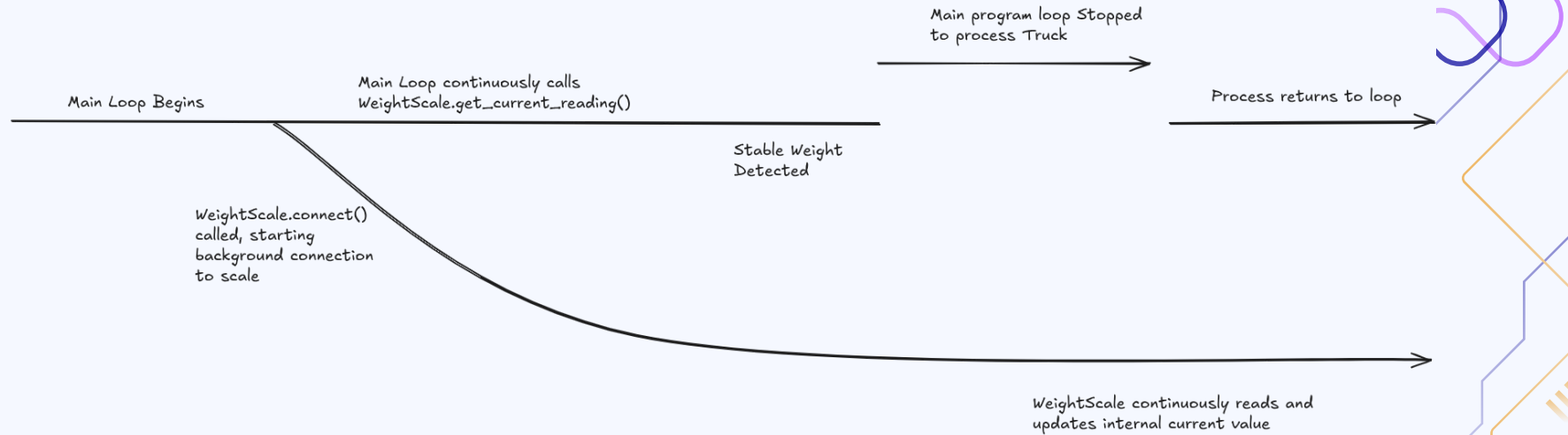


Weight Scale & Simulation

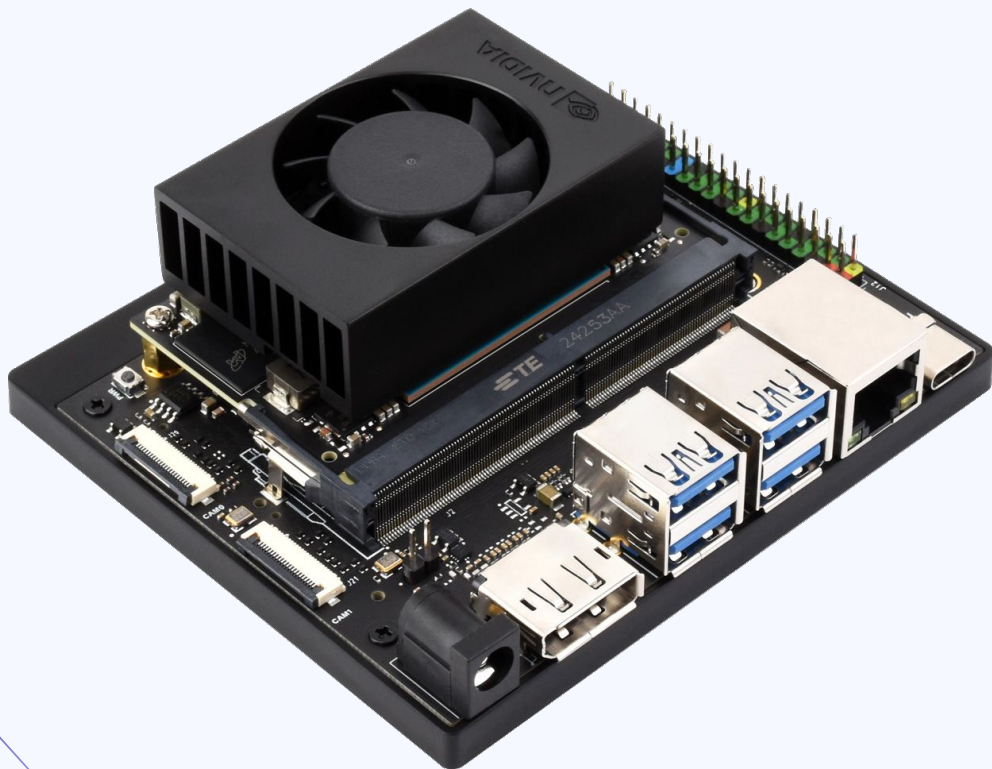
- The weight scale exposes a configurable TCP port on the network
 - Any device can connect to the port and start listening
 - We used a software simulator for project testing and demonstration
- The weight scale module operates using Python's Socket library
 - Spawns thread to constantly update latest scale value
 - Main loop continuously pulls and parses latest value
 - Starts the rest of program execution when a stable weight is detected

The screenshot shows a software interface titled "Inbound Scale". At the top, there is a large black rectangular display showing the number "000000" in red, resembling a digital scale's readout. Below this display are two buttons: "Tare" and "Key". Underneath these is a section labeled "Manual" containing a 3x3 grid of buttons with numbers 1 through 9, and two additional buttons labeled "Enter" and "0", with a "Cancel" button to the right of the "0" button. Below the "Manual" section is a "Jitter" button. At the bottom, there is a "Set Weight" section with a radio button (currently unselected) and a checked checkbox labeled "Motion". To the right of the "Set Weight" section are two buttons: "Sim Pull On" and "Sim Drive".

Weight Scale & Simulation



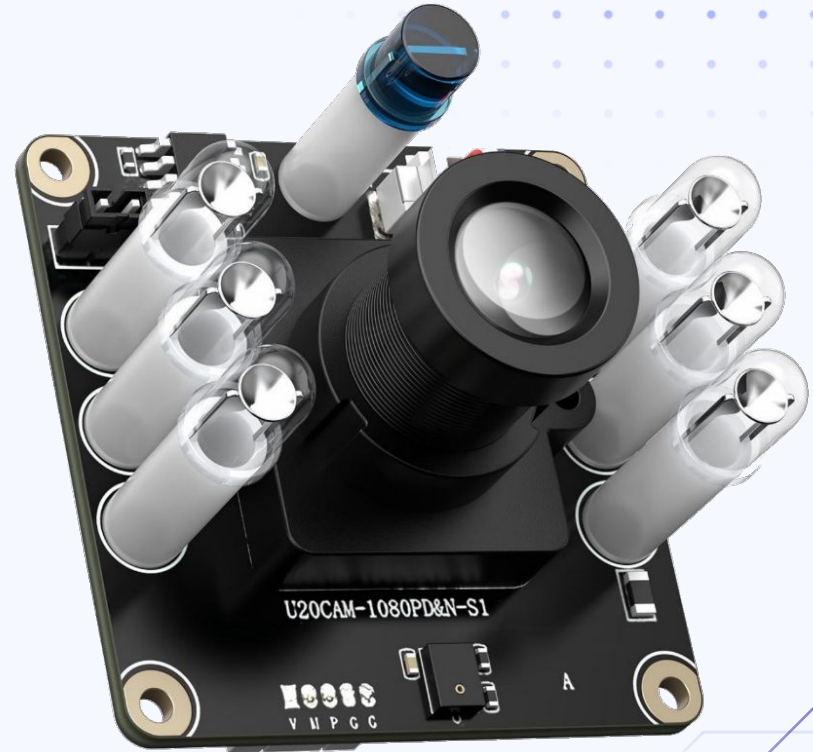
Jetson Orin Nano



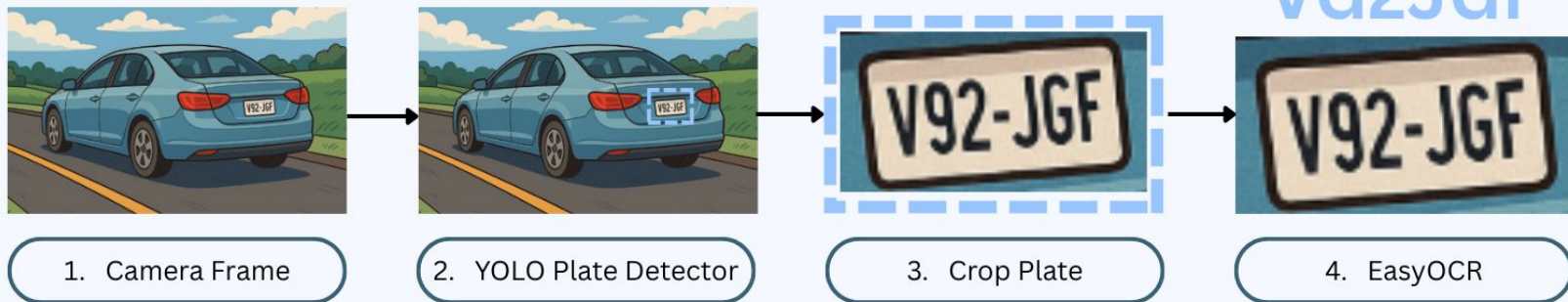
- 1024 Core GPU
 - 32 Tensor Cores
- 6 Core CPU
- 8GB Ram
- 4x USB 3.2 Ports
 - Using Only 2 for 2 cameras
- USB Type-C
- microSD card slot
- Display port

Camera Integration

- 6x Infrared LEDs
- Photosensitive Diode
- Greyscale/Color Jumper
 - Set to color for better Cargo ID
- Microphone (unused)
- Connect via USB



License Plate Recognition (LPR)



- Real-time inference on Jetson Orin Nano
- YOLOv8 trained for plate detection
- Bounding box crop isolates plate region
- EasyOCR processes cropped image for text extraction
- Plate number validated and logged with transaction data

Cargo Classification



**Cargo Load Detection
(YOLO)**



Image Classification

Model: MobileNetV3-Large
A lightweight CNN pretrained on
ImageNet (~4.2M parameters)

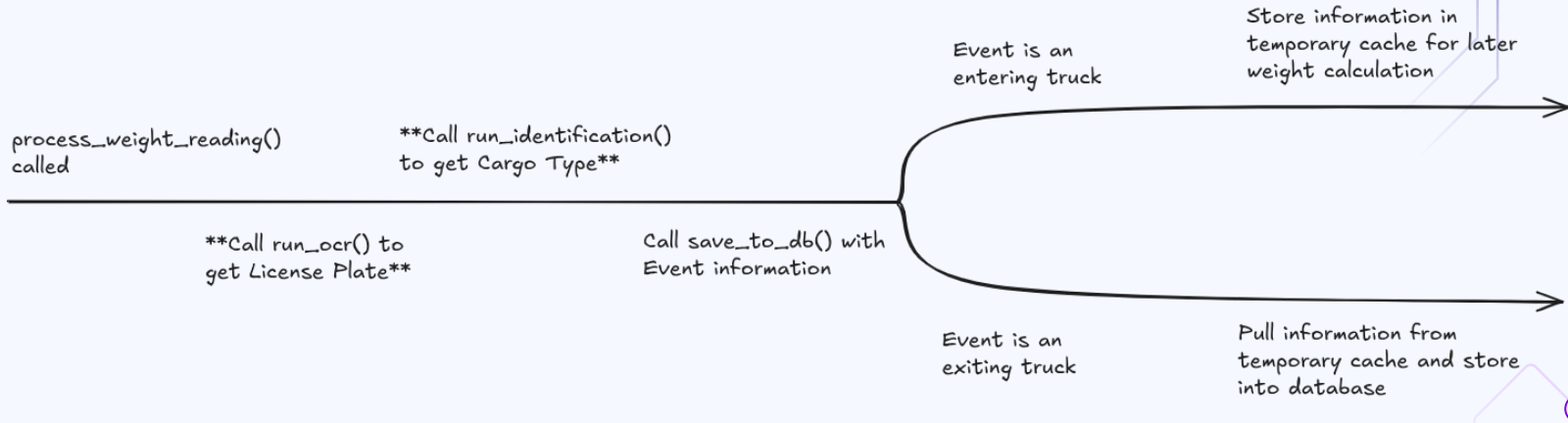


BRICKS

Model Output

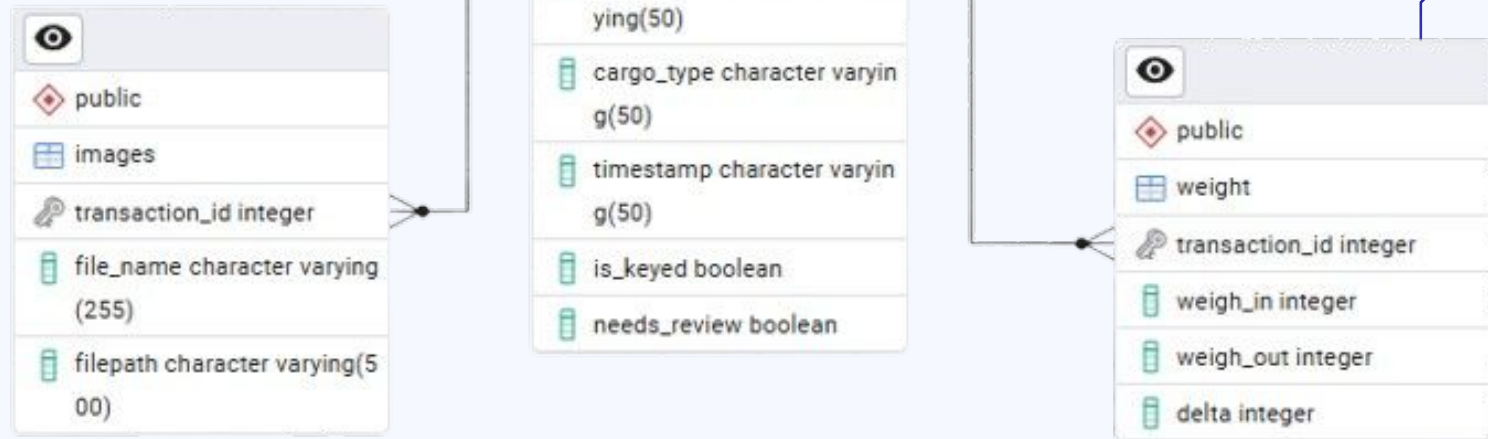
Backend

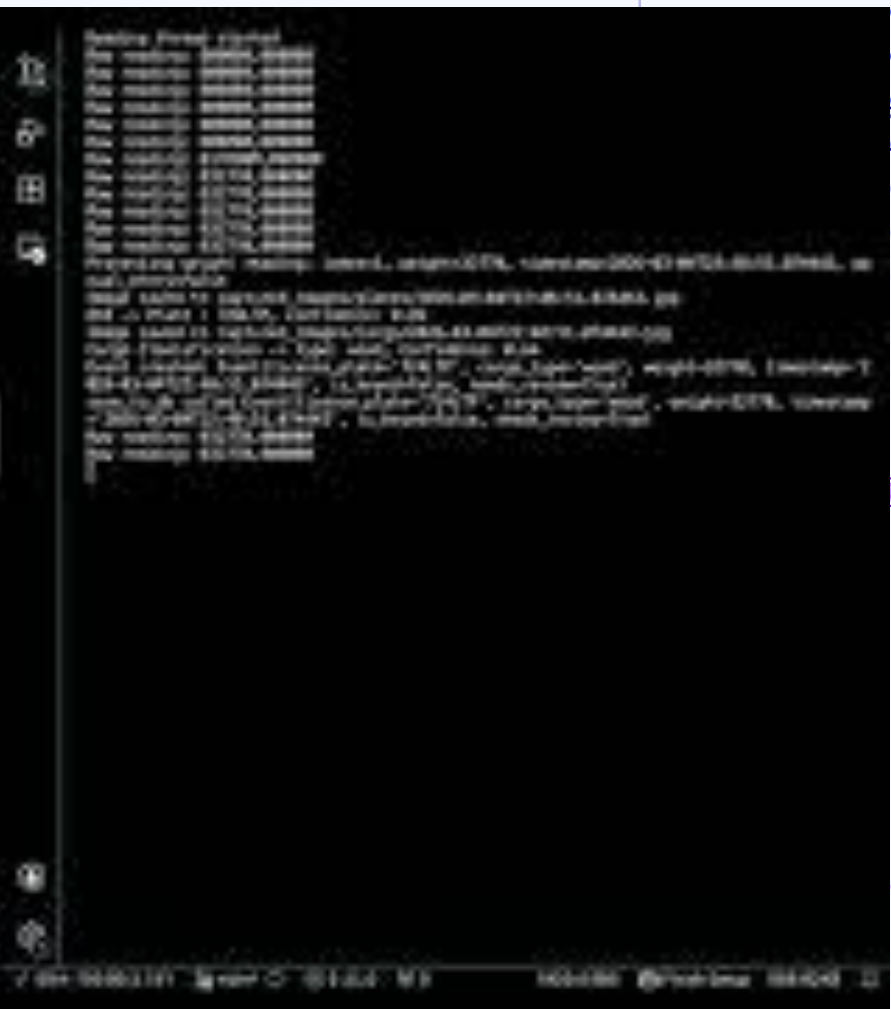
- The backend of our system is implemented as a single library file with multiple external libraries
 - It is called once from weigh_scale for every stable weight detection
 - Processes a weight event, updating internal cache or writing out to database



Database and ERD

- PostgreSQL database
- Needs to store all relevant information:
 - weight information
 - transaction id / truck information
 - truck images and review flagging





Demo Images



Cargo Image from Demo
Predicted: Wood



Plate Image from Demo
Predicted: 724LTP

Demo Database Output

```
[a38devdb-copy=# SELECT * FROM transactions;  
 id | license_plate | cargo_type |          timestamp          | is_keyed | needs_review  
-----+-----+-----+-----+-----+-----  
  1 | 724LTP       | wood      | 2026-03-04T22:48:51.074443 | f        | f  
(1 row)
```

```
[a38devdb-copy=# SELECT * FROM weight;  
 transaction_id | weigh_in | weigh_out | delta  
-----+-----+-----+-----  
           1 |   32770 |   19460 | 13310  
(1 row)
```

```
[a38devdb-copy=# SELECT * FROM images;  
 transaction_id |          file_name          |          filepath  
-----+-----+-----  
           1 | 2026-03-04T22:48:51.074443 | captured_images/cargo  
(1 row)
```

Budget

Part Name	Source	Quantity	Price
Nvidia Jetson Orin Nano Development Kit	Sponsors	1	Free
Camera	Sponsors (Amazon)	2	\$27
		GRAND TOTAL:	\$54

Successes & Difficulties

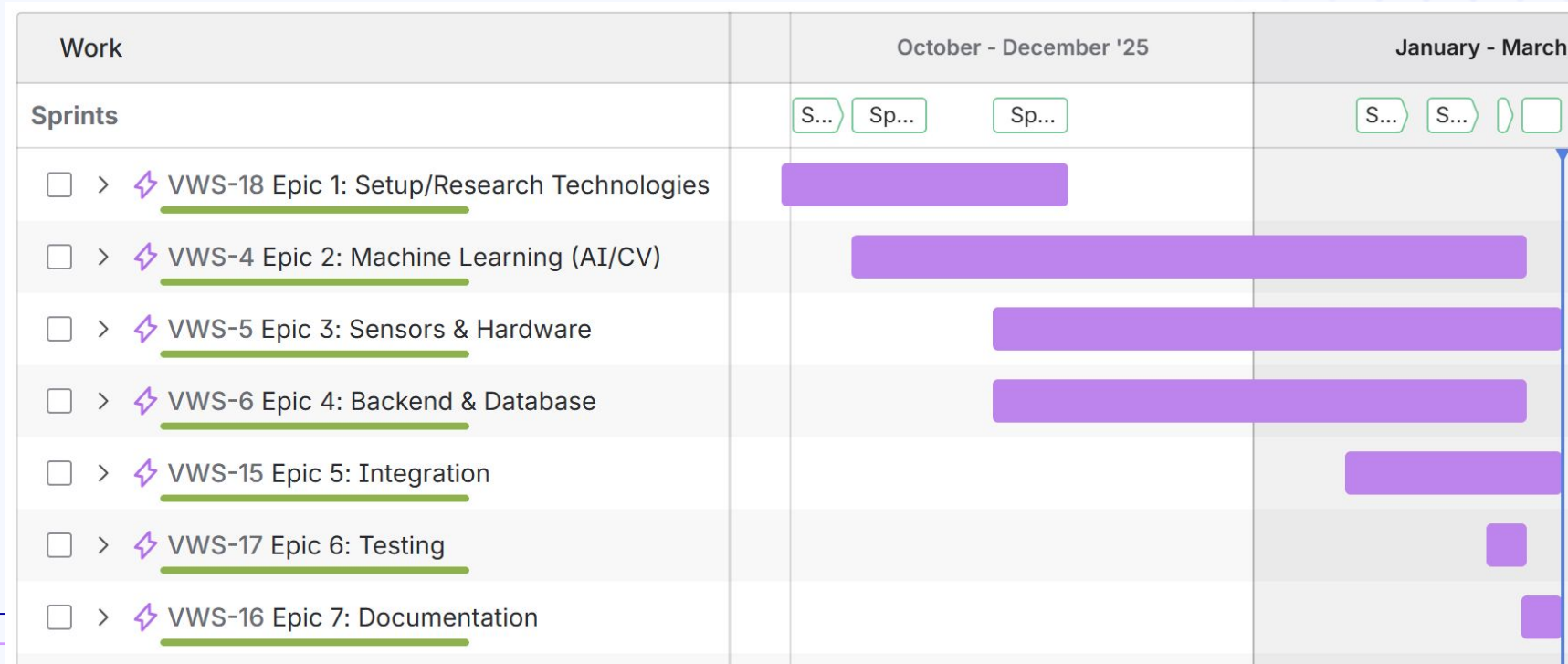
Successes

- Complete pipeline across entire process
- Confidence-based review system
- Cargo Detection has 92.34% accuracy
- Connecting to Jetson GPU
- Implementations with hardware work as expected

Difficulties

- Extreme delay on database access
- Issues with initial weight simulator
- No access to real sponsor dataset
- Real-world variation
- Decorative plate graphics (ex: Florida orange) interfere with OCR
- Variable lighting conditions can cause issues with cameras & plate ID

Jira





THANK YOU!!

Demo Time!!